CloudFormation

Introduction to CloudFormation

CloudFormation is like a blueprint for creating and managing AWS resources. Instead of manually setting up each service (like EC2, S3, etc.), you define what you want in a template file, and CloudFormation automatically builds everything for you, exactly as you describe.

Example: Imagine you need a website that has a load balancer, a couple of web servers, and storage for files. Instead of setting up each part manually, you create a CloudFormation template that says:

1. "I need a load balancer."
2. "I need two EC2 instances (virtual servers)."
3. "I need an S3 bucket to store images and files."

CloudFormation reads your template, figures out what needs to be created, and does it all in the correct order:

- First, it creates the load balancer.
- Then, it launches the two EC2 instances.
- Finally, it creates the S3 bucket for storage.

CloudFormation saves you time by doing all this automatically, ensuring everything is set up with the exact configurations you need.

Benefits of AWS CloudFormation

I**nfrastructure as Code:** CloudFormation lets you define your entire infrastructure using code. This means instead of clicking buttons and filling out forms, you can write a template that tells CloudFormation exactly what resources you need.

Example: If you need a load balancer, an EC2 instance, and an S3 bucket, you can define them all in a CloudFormation template instead of creating them one by one manually.

No Manual Resource Creation: Since CloudFormation creates the resources for you based on the code, there's no chance of mistakes from doing things manually. It gives you more control and consistency.

Example: If you're setting up multiple environments (like development and production), CloudFormation ensures that each environment is set up the same way every time, without human error.

CloudFormation

Change Review Through Code: When you need to make changes to your infrastructure, you update the template, and everyone can review those changes in the code before anything happens. This makes the process more transparent and predictable.
Example: If you want to add another EC2 instance, you just update the template. The change is clear, and everyone can see what's being changed before CloudFormation applies it

Cost Management: CloudFormation tags each resource with an identifier so you can easily see how much each resource is costing. You can also estimate costs before creating resources.
Example: If you want to know how much your EC2 instance and S3 bucket will cost, you can check the CloudFormation template's estimated cost before you launch them.

Savings Strategy: CloudFormation can help with cost-saving by automating resource creation and deletion. For example, in a development environment, you could automatically delete resources after work hours and recreate them the next day.
Example: Your company might automate CloudFormation to shut down and delete dev resources to save money, and then recreate them at next day when work starts again.

More Benefits of AWS CloudFormation

Productivity: CloudFormation boosts your productivity by automating the setup and management of your resources. Once your template is written, you don't need to manually create or configure resources over and over again.
Example: If you need to create 10 EC2 instances for a new project, CloudFormation can do it automatically with just a template, saving you tons of time.

Destroy and Re-create Infrastructure on the Fly: If you need to make changes or restart your infrastructure, CloudFormation allows you to delete and recreate resources easily. This is helpful when you need to rebuild or update environments quickly.
Example: If something goes wrong in your development environment, you can delete the entire stack and recreate it with the latest configurations in minutes.

Automated Diagram Generation: CloudFormation can automatically generate a visual diagram for your infrastructure, helping you understand how all the resources are connected.
Example: After creating your stack, you can quickly see a diagram of your EC2 instances, load balancers, and S3 buckets and how they work together.

CloudFormation

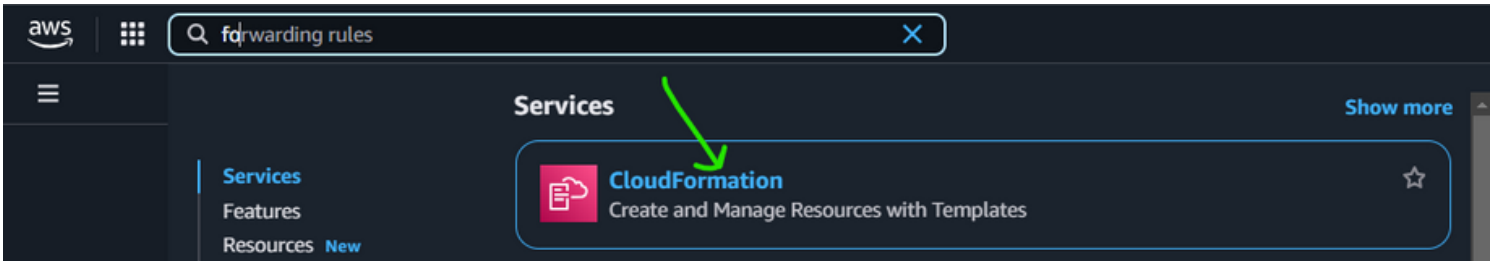
Declarative Programming: With CloudFormation, you just tell it what you want, not how to create it. You don't need to worry about the order or steps; CloudFormation handles that for you.
Example: If you need a database and an EC2 instance, CloudFormation knows which resource should be created first and does it for you without needing you to define each step.

Don't Re-invent the Wheel: CloudFormation lets you reuse existing templates, so you don't have to start from scratch every time.
Example: There are tons of templates available online for common setups (like creating a VPC, EC2, etc.). You can use these templates and adjust them to fit your needs, saving you time.

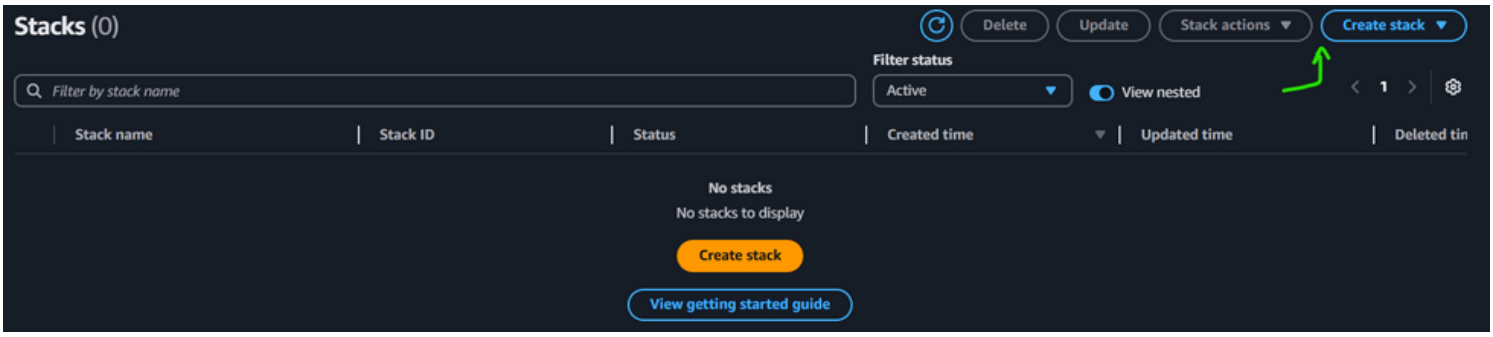
Leverage the Documentation: AWS provides detailed documentation to guide you in creating templates and understanding how CloudFormation works, making it easier to learn and use.
Example: If you're not sure how to create an S3 bucket in CloudFormation, you can look up examples in the AWS documentation to see how others have done it.

Supports Almost All AWS Resources: CloudFormation supports most of the AWS services you'll use. If there's a service that isn't supported, you can still create custom resources for it.
Example: If CloudFormation doesn't directly support a particular AWS feature, you can create a custom resource to handle it, like using Lambda functions for advanced use cases.

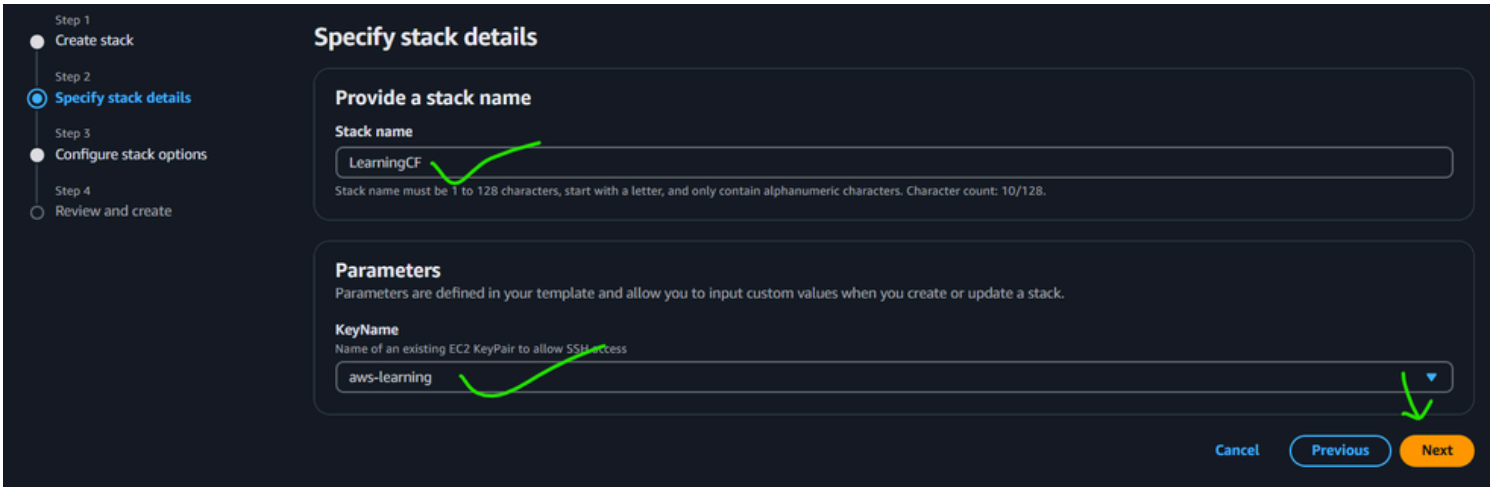
CloudFormation



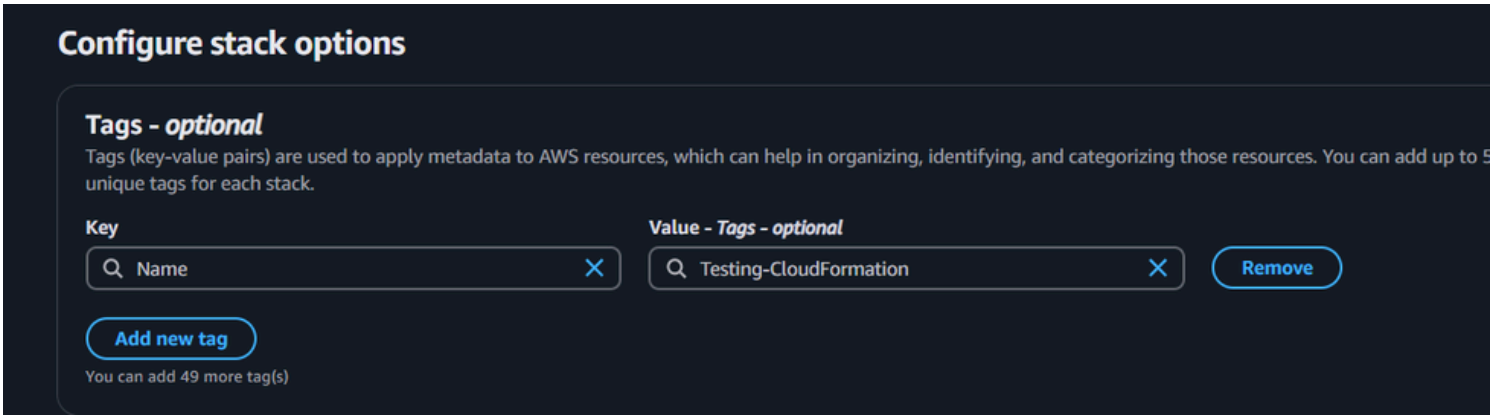
CloudFormation: Click on Search & find the Services cloudformation



Create your first stack

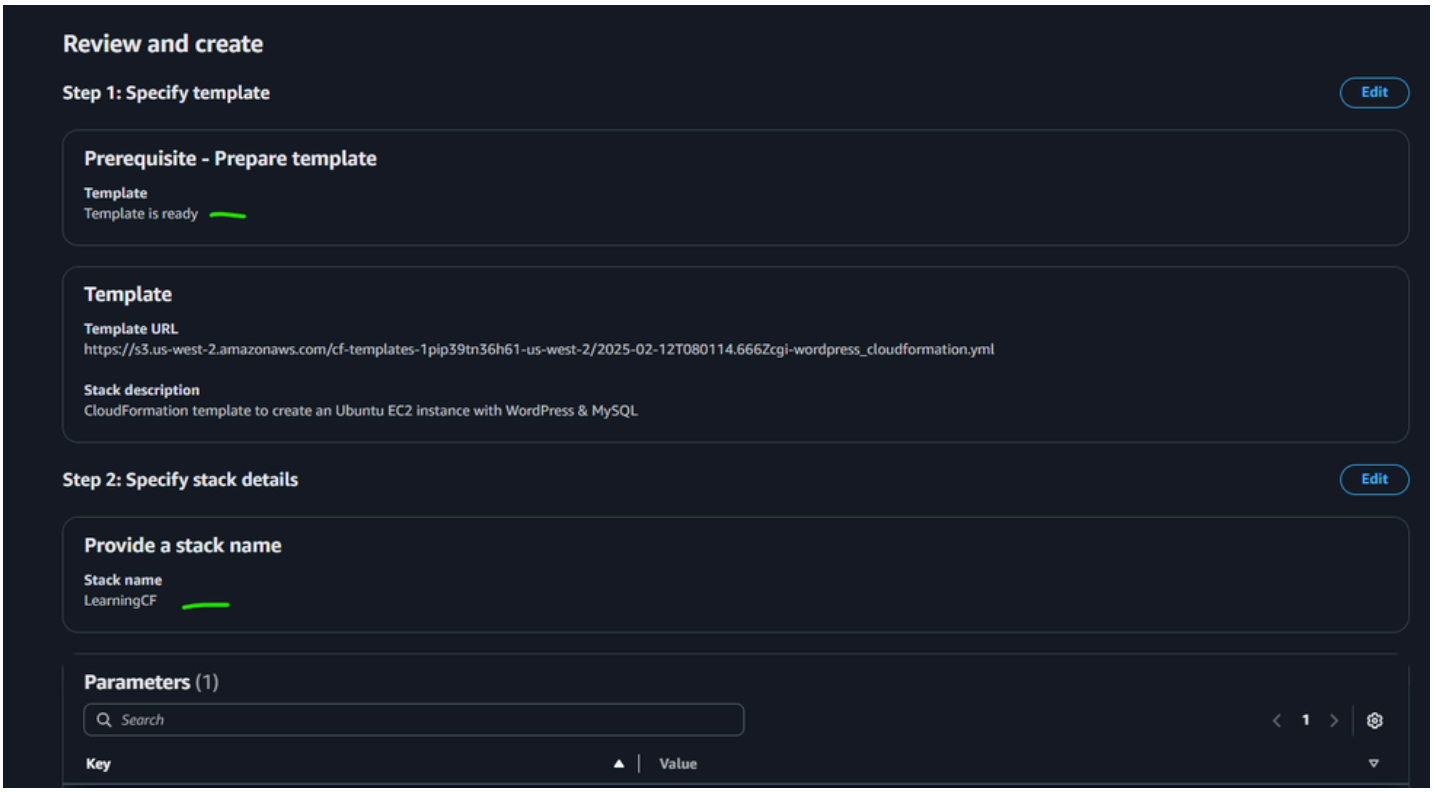


Give the stack name : LearningCF , Parameters keyname : aws-learning

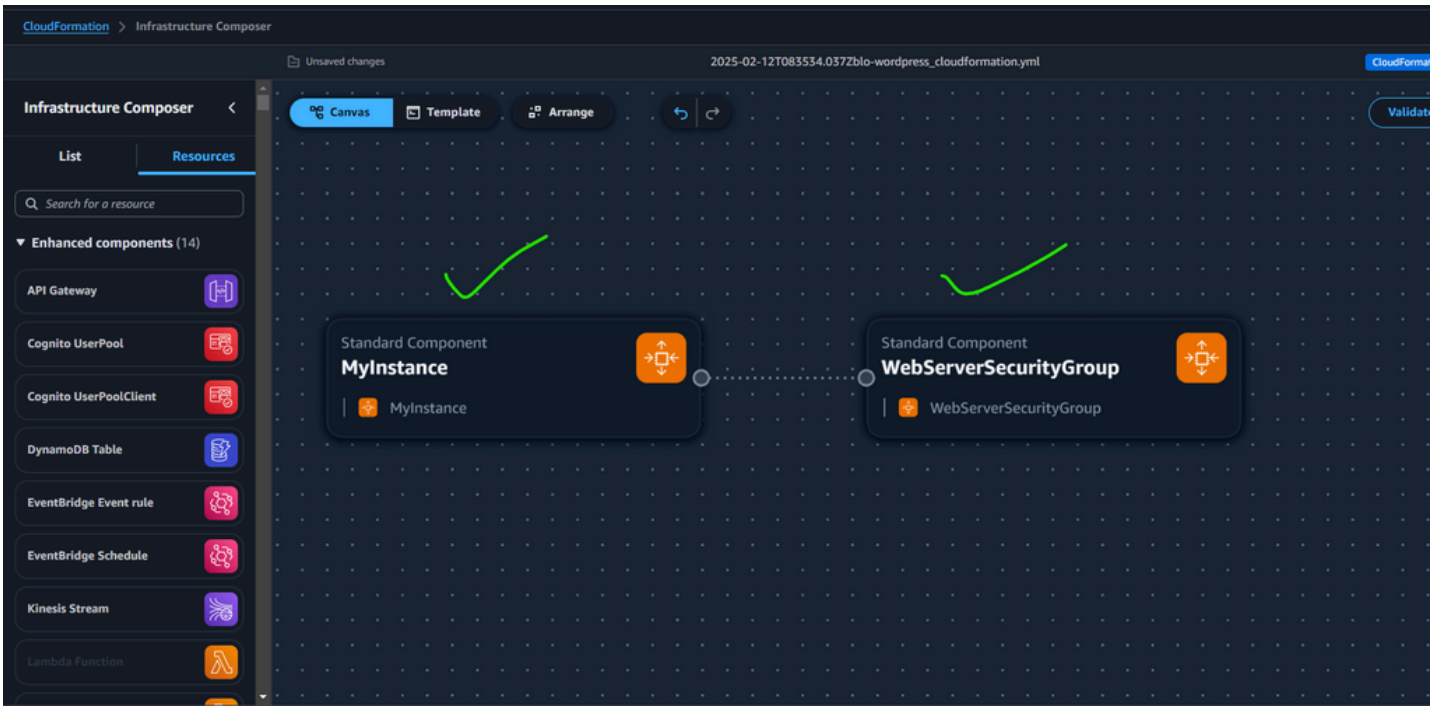


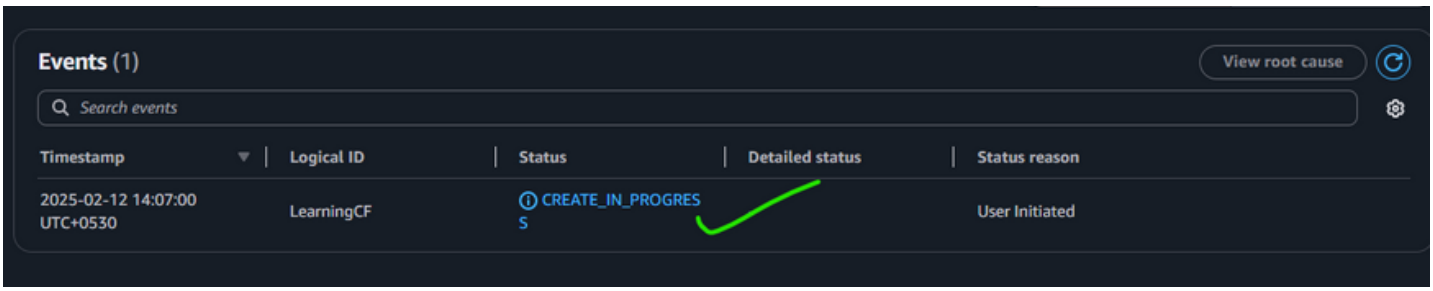
Give the Tag Name

CloudFormation



Review : Your Template





All Event's

Stack info | Events - updated | Resources | Outputs | Parameters | Template | Change sets | Git sync

Table view | Timeline view - new

Events (10)

View root cause

Search events

Timestamp	Logical ID	Status	Detailed status	Status reason
2025-02-12 14:07:27 UTC+0530	LearningCF	CREATE_COMPLETE	-	-
2025-02-12 14:07:25 UTC+0530	MyInstance	CREATE_COMPLETE	-	-
2025-02-12 14:07:25 UTC+0530	LearningCF	CREATE_IN_PROGRESS	CONFIGURATION_COMPLETE	Eventual consistency check initiated
2025-02-12 14:07:25 UTC+0530	MyInstance	CREATE_IN_PROGRESS	CONFIGURATION_COMPLETE	Eventual consistency check initiated
2025-02-12 14:07:14 UTC+0530	MyInstance	CREATE_IN_PROGRESS	-	Resource creation Initiated
2025-02-12 14:07:12 UTC+0530	MyInstance	CREATE_IN_PROGRESS	-	-
2025-02-12 14:07:11 UTC+0530	WebServerSecurityGroup	CREATE_COMPLETE	-	-
2025-02-12 14:07:05 UTC+0530	WebServerSecurityGroup	CREATE_IN_PROGRESS	-	Resource creation Initiated
2025-02-12 14:07:02 UTC+0530	WebServerSecurityGroup	CREATE_IN_PROGRESS	-	-
2025-02-12 14:07:00 UTC+0530	LearningCF	CREATE_IN_PROGRESS	-	User Initiated

Resource Created

Stack info | Events - updated | Resources | Outputs | Parameters | Template | Change sets | Git sy

Resources (2)

Search resources

Logical ID	Physical ID	Type	Status
MyInstance	i-0dc61c83c8b5898ba	AWS::EC2::Instance	CREATE_COMPLETE
WebServerSecurityGroup	LearningCF-WebServerSecurityGroup-gdrb7Wgzxlu0	AWS::EC2::SecurityGroup	CREATE_COMPLETE

Resource Template

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: 'CloudFormation template to create an Ubuntu EC2 instance with WordPress & MySQL'

Parameters:
  KeyName:
    Description: 'Name of an existing EC2 KeyPair to allow SSH access'
    Type: 'AWS::EC2::KeyPair::KeyName'

Resources:
  MyInstance:
    Type: 'AWS::EC2::Instance'
    Properties:
      InstanceType: 't2.micro'
      ImageId: 'ami-00c257e12d6828491'  # Ubuntu 22.04 LTS (Check for latest in us-west-2)
      KeyName: !Ref KeyName
      SecurityGroups:
        - !Ref WebServerSecurityGroup
      UserData:
        Fn::Base64: !Base64 |
          #!/bin/bash
          sudo apt update -y
          sudo apt install -y apache2 mysql-server php libapache2-mod-php php-mysql unzip wget
          sudo systemctl enable apache2
          sudo systemctl start apache2
          sudo systemctl enable mysql
          sudo systemctl start mysql

          # Secure MySQL Installation
          sudo mysql -e "ALTER USER 'root'@'localhost' IDENTIFIED WITH 'mysql_native_password' BY 'root';"
          sudo mysql -e "CREATE DATABASE wordpress;"
          sudo mysql -e "CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'wp_password';"
          sudo mysql -e "GRANT ALL PRIVILEGES ON wordpress.* TO 'wp_user'@'localhost';"
          sudo mysql -e "FLUSH PRIVILEGES;"

          # Download and Setup WordPress
          cd /var/www/html
          sudo wget https://wordpress.org/latest.tar.gz
          sudo tar -xzf latest.tar.gz
          sudo mv wordpress/* .
          sudo rm -rf wordpress latest.tar.gz
```

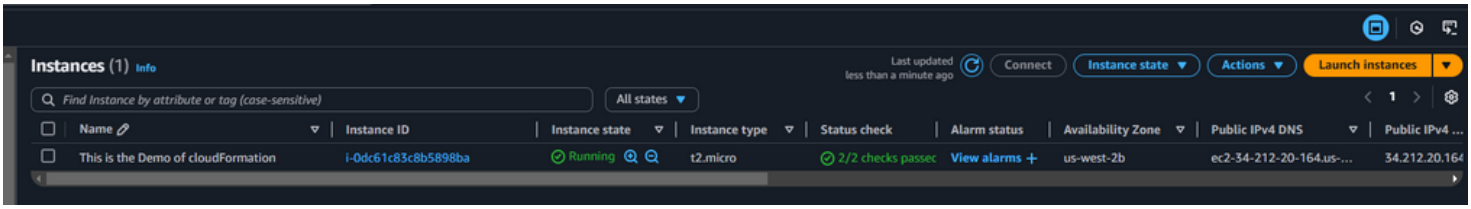
```yaml
          sudo chown -R www-data:www-data /var/www/html
          sudo chmod -R 755 /var/www/html

          # Configure WordPress Database
          sudo cp wp-config-sample.php wp-config.php
          sudo sed -i "s/database_name_here/wordpress/" wp-config.php
          sudo sed -i "s/username_here/wp_user/" wp-config.php
          sudo sed -i "s/password_here/wp_password/" wp-config.php

          # Restart Apache
          sudo systemctl restart apache2

  WebServerSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: 'Enable SSH and HTTP access'
      SecurityGroupIngress:
        - IpProtocol: 'tcp'
          FromPort: 22
          ToPort: 22
          CidrIp: '0.0.0.0/0'
        - IpProtocol: 'tcp'
          FromPort: 80
          ToPort: 80
          CidrIp: '0.0.0.0/0'

Outputs:
  WebsiteURL:
    Description: 'Public URL of WordPress'
    Value: !Sub 'http://${MyInstance.PublicDnsName}'
```

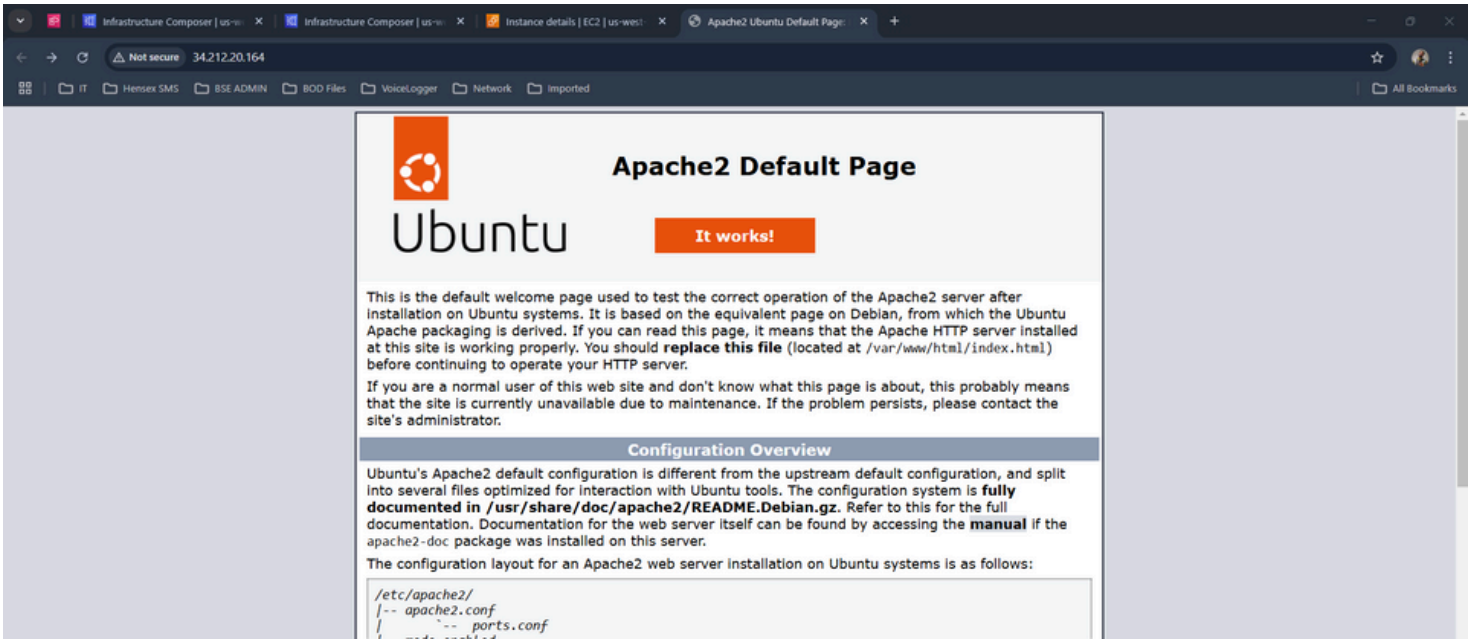
Resource Template

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'CloudFormation template to create an Ubuntu EC2 instance with WordPress & MySQL'

Parameters:
  KeyName:
    Description: 'Name of an existing EC2 KeyPair to allow SSH access'
    Type: 'AWS::EC2::KeyPair::KeyName'

Resources:
  MyInstance:
    Type: 'AWS::EC2::Instance'
    Properties:
      InstanceType: 't2.micro'
      ImageId: 'ami-00c257e12d6828491'  # Ubuntu 22.04 LTS (Check for latest in us-west-2)
      KeyName: !Ref KeyName
      SecurityGroups:
        - !Ref WebServerSecurityGroup
      UserData:
        Fn::Base64: !Base64 |
          #!/bin/bash
          sudo apt update -y
          sudo apt install -y apache2 mysql-server php libapache2-mod-php php-mysql unzip wget
          sudo systemctl enable apache2
          sudo systemctl start apache2
          sudo systemctl enable mysql
          sudo systemctl start mysql

          # Secure MySQL Installation
          sudo mysql -e "ALTER USER 'root'@'localhost' IDENTIFIED WITH 'mysql_native_password' BY 'root';"
          sudo mysql -e "CREATE DATABASE wordpress;"
          sudo mysql -e "CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'wp_password';"
          sudo mysql -e "GRANT ALL PRIVILEGES ON wordpress.* TO 'wp_user'@'localhost';"
          sudo mysql -e "FLUSH PRIVILEGES;"

          # Download and Setup WordPress
          cd /var/www/html
          sudo wget https://wordpress.org/latest.tar.gz
          sudo tar -xzf latest.tar.gz
          sudo mv wordpress/* .
          sudo rm -rf wordpress latest.tar.gz
          sudo chown -R www-data:www-data /var/www/html
          sudo chmod -R 755 /var/www/html

          # Configure WordPress Database
          sudo cp wp-config-sample.php wp-config.php
          sudo sed -i "s/database_name_here/wordpress/" wp-config.php
          sudo sed -i "s/username_here/wp_user/" wp-config.php
          sudo sed -i "s/password_here/wp_password/" wp-config.php

          # Restart Apache
          sudo systemctl restart apache2

  WebServerSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: 'Enable SSH and HTTP access'
      SecurityGroupIngress:
        - IpProtocol: 'tcp'
          FromPort: 22
          ToPort: 22
          CidrIp: '0.0.0.0/0'
        - IpProtocol: 'tcp'
          FromPort: 80
          ToPort: 80
          CidrIp: '0.0.0.0/0'

Outputs:
  WebsiteURL:
    Description: 'Public URL of WordPress'
    Value: !Sub 'http://${MyInstance.PublicDnsName}'
```

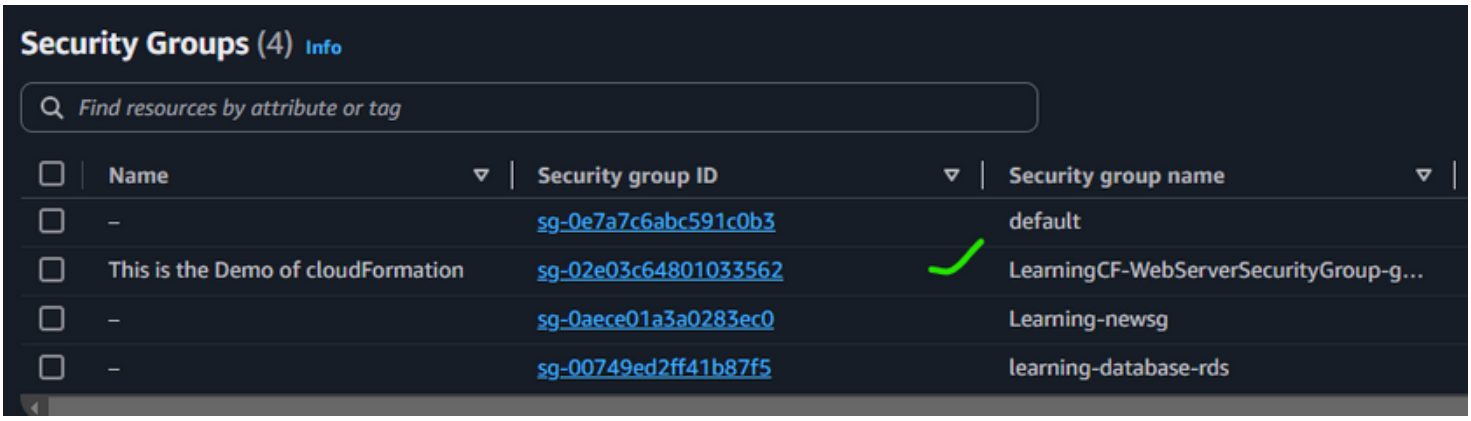
•

Resource Created





Security Group Created



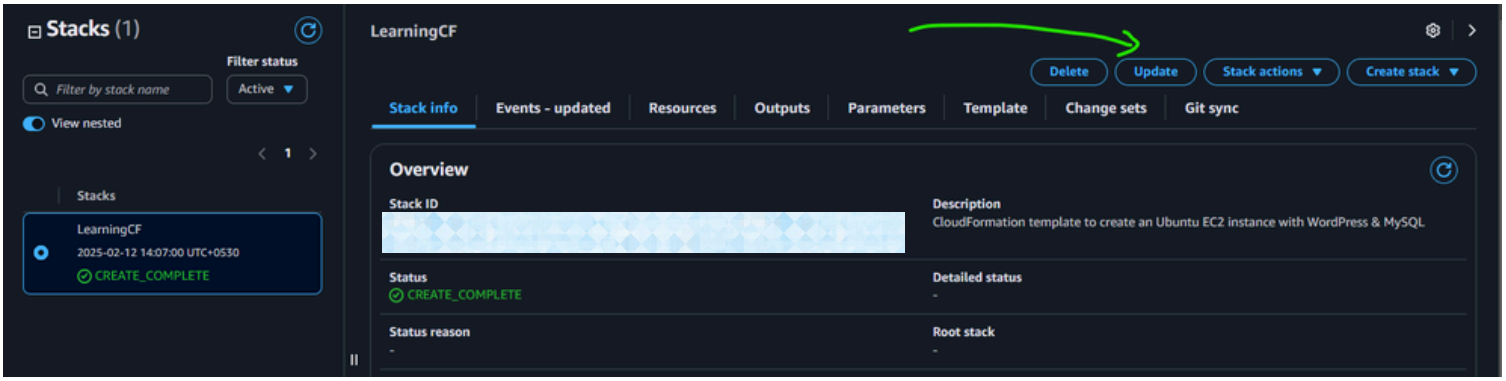
Update With New Resources

- In New YML file we have remove all the configuration & install nginx only

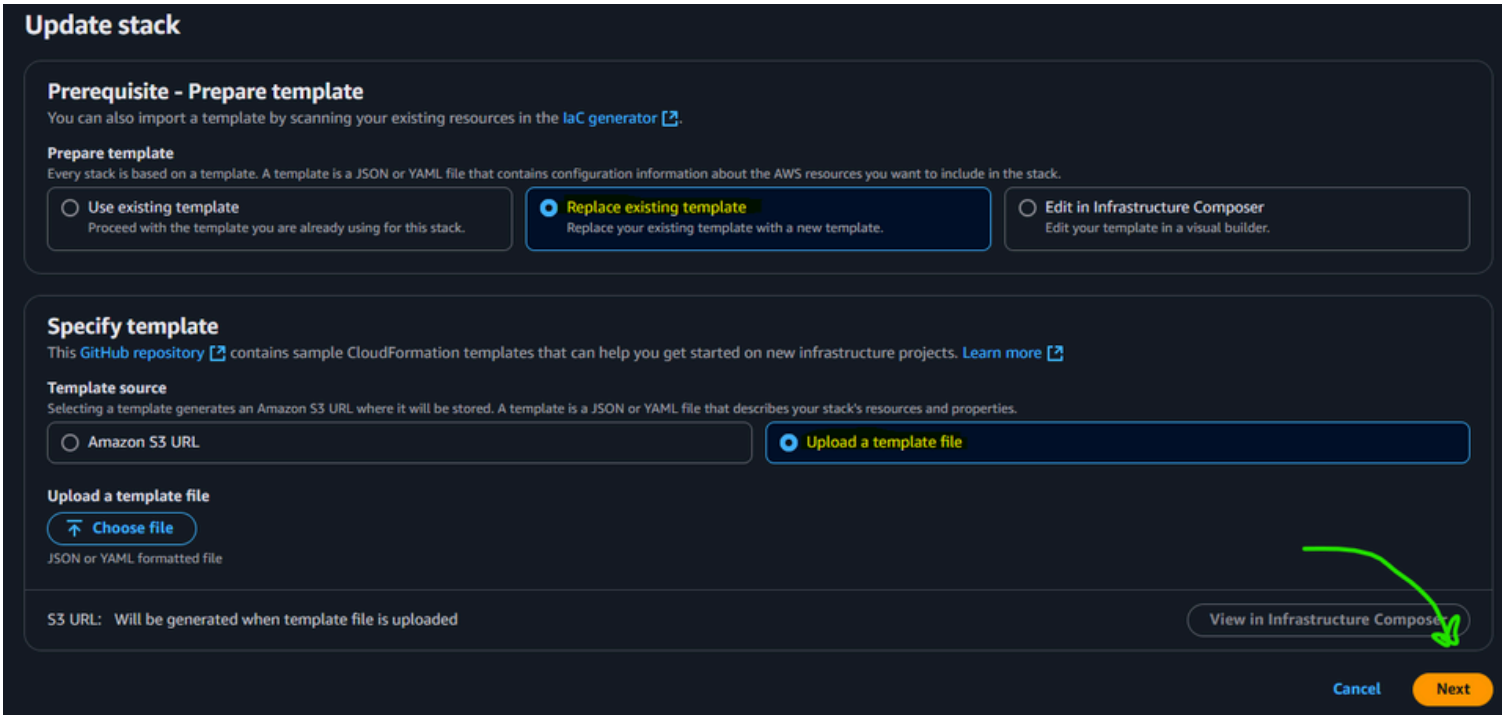
```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: 'CloudFormation template to create an Ubuntu EC2 instance with Nginx'

Parameters:
  KeyName:
    Description: 'Name of an existing EC2 KeyPair to allow SSH access'
    Type: 'AWS::EC2::KeyPair::KeyName'

Resources:
  MyInstance:
    Type: 'AWS::EC2::Instance'
    Properties:
      InstanceType: 't2.micro'
      ImageId: 'ami-00c257e12d6828491'  # Ubuntu 22.04 LTS (Check for latest in us-
      KeyName: !Ref KeyName
      SecurityGroups:
        - !Ref WebServerSecurityGroup
      UserData:
        Fn::Base64: !Base64 |
          #!/bin/bash
          sudo apt update -y
          sudo apt install -y nginx
          sudo systemctl enable nginx
          sudo systemctl start nginx
```

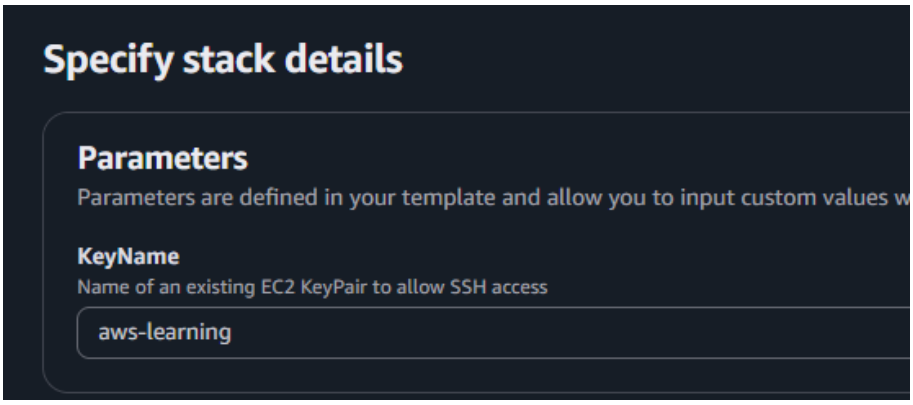
```yaml
  WebServerSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: 'Enable SSH and HTTP access'
      SecurityGroupIngress:
        - IpProtocol: 'tcp'
          FromPort: 22
          ToPort: 22
          CidrIp: '0.0.0.0/0'
        - IpProtocol: 'tcp'
          FromPort: 80
          ToPort: 80
          CidrIp: '0.0.0.0/0'

tputs:
  WebsiteURL:
    Description: 'Public URL of Nginx Server'
    Value: !Sub 'http://${MyInstance.PublicDnsName}'
```

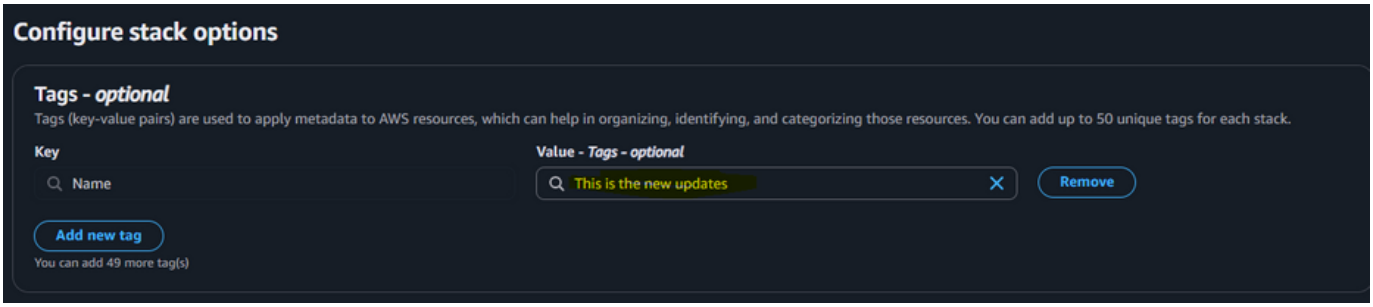
Update



Click on Update : Upload New Update configuration.yml file







Update

Click on Update : Update New Resources & terminate old instance & SG

Action	Logical ID	Physical ID	Resource type	Replacement
Modify	MyInstance		AWS::EC2::Instance	Conditional
Modify	WebServerSecurityGroup		AWS::EC2::SecurityGroup	False

Events (11)

Search events

Timestamp	Logical ID	Status	Detailed status	Status reason
2025-02-12 14:34:50 UTC+0530	LearningCF	UPDATE_IN_PROGRESS	-	User Initiated

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Cloudformation template to create an Ubuntu EC2 instance with Nginx'

Parameters:
  KeyName:
    Description: 'Name of an existing EC2 KeyPair to allow SSH access'
    Type: 'AWS::EC2::KeyPair::KeyName'

Resources:
  MyInstance:
    Type: 'AWS::EC2::Instance'
    Properties:
      InstanceType: 't2.micro'
      ImageId: 'ami-00c257e12d6828491'  # Ubuntu 22.04 LTS (Check for latest in us-west-2)
      KeyName: !Ref KeyName
      SecurityGroups:
        - !Ref WebServerSecurityGroup
      UserData:
        Fn::Base64: !Base64 |
          #!/bin/bash
          sudo apt update -y
          sudo apt install -y nginx
          sudo systemctl enable nginx
          sudo systemctl start nginx

  WebServerSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: 'Enable SSH and HTTP access'
      SecurityGroupIngress:
        - IpProtocol: 'tcp'
          FromPort: 22
          ToPort: 22
          CidrIp: '0.0.0.0/0'
        - IpProtocol: 'tcp'
          FromPort: 80
          ToPort: 80
          CidrIp: '0.0.0.0/0'

Outputs:
  WebsiteURL:
    Description: 'Public URL of Nginx Server'
    Value: !Sub 'http://${MyInstance.PublicDnsName}'
```

Instances (1) Info

Last updated less than a minute ago

Find Instance by attribute or tag (case-sensitive) | All states

Name	Instance ID	Instance state	Instance type
This is the new updates	i-0dc61c83c8b5898ba	Running	t2.micro

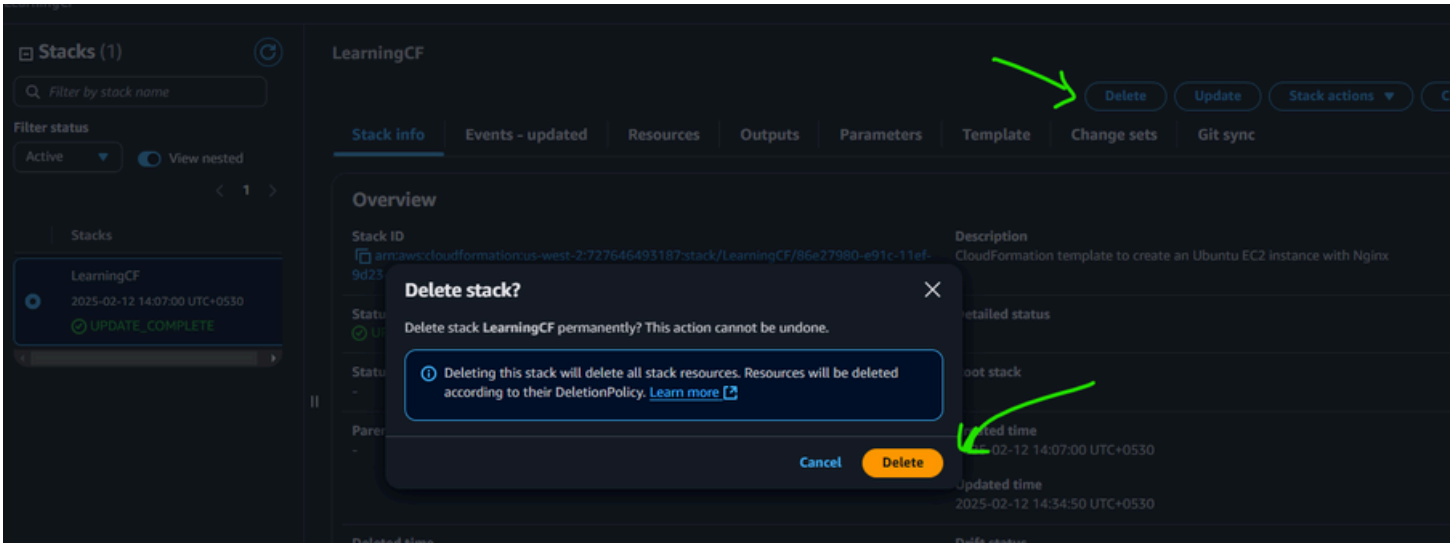
Security Groups (4) Info

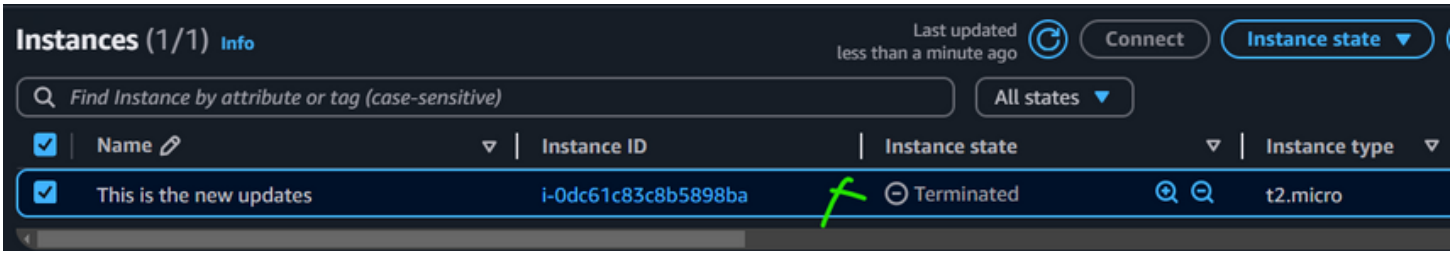
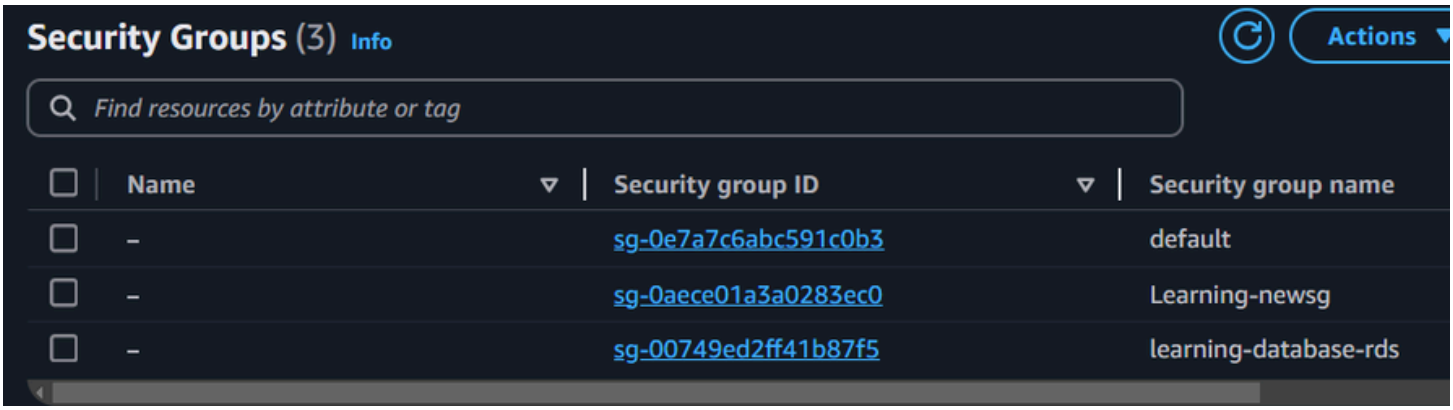
Find resources by attribute or tag

Name	Security group ID	Security group name
-	sg-0e7a7c6abc591c0b3	default
This is the new updates	sg-02e03c64801033562	LearningCF-WebServerSecurityGroup-g...
-	sg-0aece01a3a0283ec0	Learning-newsg
-	sg-00749ed2ff41b87f5	learning-database-rds

Delete

Delete: Delete all Running configuration with is created by CloudFormation





Delete: No Security Group & Instance Available after Delete all resources delete automatically one by one